

Vision Day04 hw1 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.25

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에선 ROS2 로 카메라 영상을 받아 YOLO 모델로 물체를 탐지하고, 탐지 박스와 클래스 번호·이름·점수를 화면에 표시하는 프로그램을 구성하였다. 카메라 노드와 추론 노드를 분리하고, 두 노드가 이미지 토픽으로 데이터를 주고받도록 했다. 사용한 모델은 yolo26n.pt 이며, CPU 에서 PyTorch 와 Ultralytics 로 실행하였다.

AI 는 과제내용을 이해하고 ROS 2 패키지 구조, YAML 설정, 노드 코드, 실행 방법을 정리하는 데 사용했다. 나는 패키지 구조와 YAML 설정 도중 왜 이렇게 구성하였는지 또한 코드관련질문을 한 후 이해하였다. 또한 이후 yaml 파일에서 필요한 부분을 알맞게 수정하였다.

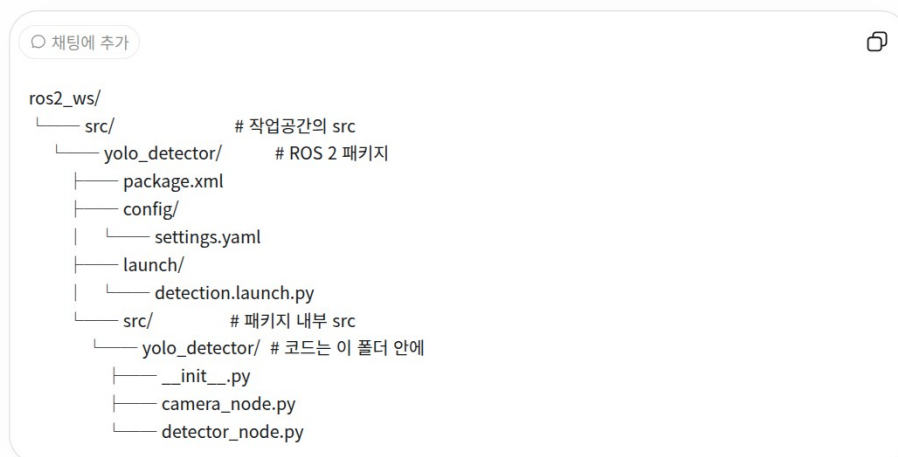
2. AI 사용 부분

처음에는 YAML 에 Hz, 토픽, 클래스 ID 와 이름, confidence 를 넣어야 한다는 조건과 패키지 내부 src 폴더 아래에 다시 하위 폴더를 두어야 한다는 조건을 AI 와 정리했다. AI 는 camera_node.py 가 카메라 영상을 읽어 /camera/image_raw 로 발행하고, detector_node.py 가 그 영상을 받아 추론하도록 패키지를 구성했다. config/settings.yaml 에서 입력 장치, 발행·추론·미리보기 주기, 사용할 클래스, 최소 탐지 점수 등을 바꿀 수 있게 했다.

launch/detection.launch.py 로 두 노드를 함께 실행한다. 추론 노드는 모델을 시작할 때 한 번 읽고, 새 카메라 프레임이 들어올 때마다 model.predict()를 호출한다. 탐지 결과는 박스가 그려진 이미지 토픽과 클래스·좌표·점수를 담은 정보 토픽으로 보낸다.

아, 그러면 패키지 내부의 **src** 아래에 폴더를 한 단계 더 만들고, 그 안에 코드 파일을 넣으라는 뜻으로 이해하면 되겠네.

현재 조건은 **ROS 2 Jazzy + PyTorch + YOLO26n**, 코드 배치는 아래처럼 정리할 수 있어. 폴더 이름은 아직 예시야.



즉, 패키지의 **src** 바로 아래에는 코드 파일을 두지 않고 하위 폴더에 모으는 구조야. 이 배치에 맞춰 ROS 2에서 실행되도록 패키지 설정도 해주면 돼.

다음은 YAML 조건을 확인하자.

1. **Hz**: 카메라 발행 주기와 추론 주기 중 어느 것을 말했어? 구분한 설명이 없었다면 둘 다 따로 설정할 수 있게 하자.
2. **클래스**: COCO 80개 전체를 탐지하면 돼, 아니면 사람·공 같은 특정 대상만 골라야 해?
3. **토픽 이름과 confidence 기준값**: 지정받은 값이 있어, 아니면 우리가 정하면 돼?

🗨️ 🔄 📄 (수요일) 오후 7:45

Hz는 의미가 불명확하니 **카메라 발행 주기와 추론 주기를 각각 설정**하는 안이야. CPU가 느리면 실제 추론 속도는 목표 10 Hz보다 낮을 수 있어.

YAML은 다음처럼 구성하면 이해하기 쉬워. **키 이름과 값은 제안이며, 아직 파일을 만들지는 않았어.**

```
</> YAML

camera_node:
  ros__parameters:
    camera_index: 0
    publish_hz: 30.0
    image_topic: "/camera/image_raw"

detector_node:
  ros__parameters:
    inference_hz: 10.0
    image_topic: "/camera/image_raw"
    result_image_topic: "/yolo/image"
    confidence_threshold: 0.5

    # 우리가 정할 동작: true이면 모든 클래스 사용
    detect_all_classes: true

    # false로 바꾸면 아래 ID에 해당하는 클래스만 사용
    class_ids: [0, 32]
    class_names: ["person", "sports ball"]
```

여기서 0 과 32 는 **모델의 클래스 인덱스**야. 예시는 사람과 공이고, 선배 코드의 7~8개 대상을 확인하면 목록만 바꾸면 돼. 클래스 이름은 모델의 이름표와 일치하는지 검사하도록 하자.

또한 화면의 ID는 일단 **클래스 ID로 잡는** 안이야. 사람마다 번호를 붙여 계속 따라가는 **추적 ID**까지 필요하다는 조건은 아직 없으므로, 둘을 구분해두면 돼.

📄 🗨️ ↩️ (수요일) 오후 7:47

처음 실행한 화면이 10 FPS 에도 미치지 못해 AI 에게 원인을 질문했다. 측정 결과 기존 설정의 탐지 결과는 약 6.6 FPS 였다. 카메라 버퍼를 1 개로 제한한 코드가 프레임을 떨어뜨리는 것이 확인되어 2 개로 수정했고, 카메라 읽기·추론·미리보기 갱신을 분리했다. 추론 목표 주기도 10 Hz 에서 30 Hz 로 변경했다.

수정 후 같은 컴퓨터의 실제 카메라로 측정했을 때 추론 결과는 약 23.8 FPS, 미리보기는 약 26.9 FPS 였다.

3. 깨달은 점

이번 과제를 하며 모델을 불러와 predict()를 호출하면 신경망 계산이 실행된다는 점을 이해했다. predict()는 PyTorch 자체의 함수가 아니라 Ultralytics 가 제공하는 기능이고, 내부 신경망 연산은 PyTorch 가 수행한다. Ultralytics 가 전처리와 출력 해석까지 맡아주기 때문에 비교적 코드를 간단하게 처리할 수 있었다.

영상이 끊겼을 때 처음에는 CPU 스레드 수나 Hz 값을 올리면 해결될 것으로 생각했지만, 실제로는 카메라 버퍼와 영상 처리 흐름을 확인해야 했다. 4 스레드에서 8 스레드로 바꾼 효과는 뚜렷하지 않았고, 60 Hz 로 값을 높여도 카메라가 제공하는 새 프레임과 한 번의 추론에 필요한 시간이 제한이 된다는 것을 알게 되었다.

